ROADRESQ
UX State & Edge-Case Document

Version 1.0 • Failure States, Recovery Paths & Exceptional User Journeys

Scope: Customer • Provider • Admin • Booking • Matching • GPS • Network • Payments • Realtime
Objective: Ensure RoadResQ remains understandable and safe when the normal happy path breaks

DETECT → EXPLAIN → RECOVER → PROTECT STATE → CONTINUE

Document Control

Field	Value
Document	UX State & Edge-Case Document
Product	RoadResQ
Version	1.0
Status	Production UX resilience baseline
Audience	Product, UX, frontend/mobile, backend, QA, support and operations
Related documents	SRS • UI/UX Design System • Wireframe Specification • API Specification • Testing & QA Plan • Security Threat Model

1. Purpose

RoadResQ operates in environments where users may have poor connectivity, weak GPS, stressful roadside conditions, unavailable providers, payment interruptions and rapidly changing booking states. This document defines how the product should behave in exceptional states so users understand what happened, what is safe, and what they can do next.

2. Edge-Case UX Principles

• Never leave the user guessing whether an action succeeded.

• Never create a duplicate booking because a button was tapped twice or a network retry occurred.

• Never show payment success until the authoritative payment result is confirmed.

• Never lose important form data because a temporary network failure occurred.

• Always provide a recovery action or safe next step.

• Preserve the last known authoritative booking state when realtime data is unavailable.

• For emergency situations, keep safety guidance visible and avoid unnecessary friction.

• Explain failures in human language, not internal error codes.

• Do not hide operational limitations behind fake loading states.

3. State Taxonomy

State class	Meaning	UX treatment
Normal	Expected operation	Standard UI
Loading	Data/action in progress	Skeleton/progress + context
Pending	Waiting for external/system event	Status + expected next action
Degraded	Feature partially works	Explain limitation + fallback
Recoverable error	Action failed but retry is safe	Message + retry/edit
Blocked	User cannot continue until condition changes	Reason + alternative
Critical	Safety, integrity or financial risk	Strong warning + controlled action
Offline	Network unavailable	Preserve state + offline indicator
Expired	Action/session no longer valid	Refresh/re-auth/retry
Cancelled	Operation intentionally ended	Reason + next option

4. Global Error Message Pattern

Every recoverable error should follow: **WHAT HAPPENED** → **IMPACT** → **WHAT TO DO NEXT**.

Poor	Preferred
Error occurred.	We couldn't submit your assistance request. Check your connection and try again.
GPS error.	We can't get your current location. Turn on location or place the pin manually.
Payment failed.	Your payment wasn't completed. No success has been recorded. Try again or choose another method.
No provider.	No nearby provider is available right now. Try expanding the search or choose a garage.

5. No Provider Available

Trigger: Matching completes without a suitable verified/available provider.

UX: Matching status → no-provider state → clear reason → [Try Again] → [Expand Search] → [Find Garage] → [Contact Support].

• Do not display an empty map without explanation.

• Preserve vehicle, problem and location details.

• Allow search-radius expansion when supported.

• Offer scheduled/nearby garage alternatives when appropriate.

• Do not repeatedly auto-submit requests without user awareness.

State: NO_MATCH → user chooses retry/expand/alternative/cancel.

6. Provider Cancels After Acceptance

Trigger: Provider accepted the job but cancels before arrival.

UX: Immediate notification → explain provider cancellation → return to matching → show replacement search status.

• Preserve the original request; do not force the customer to re-enter details.

• Show whether a new provider is being searched automatically.

• If the user must approve a new provider/price, make that decision explicit.

• If no replacement is found, offer alternatives and support.

• Record cancellation reason internally; do not expose unnecessary provider-sensitive information.

7. Customer Cancels

Trigger: Customer attempts cancellation during an active booking.

UX: Cancellation confirmation → show applicable fee/policy → [Confirm Cancellation] / [Keep Booking].

• Cancellation rules depend on booking state.

• Do not surprise the user with a fee after cancellation.

• Once cancellation is confirmed, update all clients consistently.

• Prevent stale UI from showing the booking as active.

8. Provider Rejects Request

Trigger: Provider declines an incoming request.

UX: Customer generally should not be shown every individual rejection; matching should continue. Provider sees confirmation that request was declined.

• Matching service retries according to dispatch policy.

• Avoid exposing sensitive provider performance data to customers.

• If rejection rate creates supply failure, surface a no-match/expanded-search state.

9. Provider Request Times Out

Trigger: Provider does not accept within the configured response window.

UX: Request expires → provider no longer receives active offer → matching continues.

• Customer remains in matching state if suitable alternatives exist.

• Provider sees expired state if they return to the request.

• No duplicate acceptance should be possible after timeout.

10. GPS Unavailable

Triggers: Location permission denied, GPS disabled, poor accuracy or location service unavailable.

UX: Location screen → explain issue → [Enable Location] → [Search Address] → [Place Pin Manually].

• Do not block assistance if manual location is sufficient.

• Display location accuracy when available.

• Require confirmation of the selected location before dispatch.

• If location is too uncertain for safe matching, explain why confirmation is needed.

11. Location Permission Denied

Trigger: User explicitly denies location permission.

UX: Permission explanation → system settings guidance where appropriate → manual location fallback.

• Do not repeatedly prompt without a meaningful reason.

• Never imply that GPS is mandatory if manual location can work.

• If the user later grants permission, refresh location safely.

12. Provider GPS Unavailable

Trigger: Provider's device stops sending location during an active job.

UX: Customer sees last known location + freshness indicator + 'Live location temporarily unavailable'.

• Do not animate a stale marker as if it were live.

• Keep booking status available through normal APIs.

• Provide contact/support option if tracking remains unavailable.

• Provider should see a clear prompt to restore location permission/connectivity.

13. Network Loss During Request

Trigger: Network disappears while submitting an emergency request.

UX: Preserve all entered data → show offline banner → disable unsafe duplicate submission → retry when connection returns.

• Use an idempotency key for the request submission.

• After reconnecting, check server status before allowing another create action.

• If request was actually created, show it rather than creating a second request.

14. Network Loss During Tracking

Trigger: Customer loses internet while provider is en route.

UX: Keep last known booking status/map → show offline state → [Retry] → reconcile when online.

• Do not cancel the booking automatically.

• Show timestamp/freshness for last known provider location.

• On reconnect, fetch authoritative booking state before displaying current actions.

15. Network Loss During Estimate Approval

Trigger: User taps Approve Estimate and connection drops.

UX: Show 'Checking approval status…' rather than immediate failure → query server → display approved/not approved.

• Use idempotency to avoid duplicate approval.

• Do not allow repeated approval to create multiple state transitions.

• If status cannot be confirmed, clearly mark it as pending verification.

16. Duplicate Emergency Request

Triggers: Double tap, browser refresh, mobile retry, duplicate API call or delayed network response.

UX: Detect duplicate → keep one active request → show existing request status.

• Disable/lock primary submit action during immediate processing.

• Use server-side idempotency keys as the authoritative protection.

• If a duplicate is detected, redirect to the existing booking/request rather than showing a generic error.

17. Duplicate Payment Attempt

Trigger: User taps Pay multiple times or retries after uncertain gateway response.

UX: Payment processing state → verify transaction → show final state.

• Use payment idempotency keys.

• Never assume a failed UI response means the gateway did not receive the payment.

• Check transaction/reference status before allowing another capture attempt.

• If status is unknown, show 'Payment status is being verified' rather than failure.

18. Payment Failed

UX: Payment failure screen → amount → reason category if safe → [Retry] → [Choose Another Method] → [Contact Support].

• Never show a success state before authoritative confirmation.

• Preserve invoice and booking information.

• Retry must be safe and idempotent.

• Do not duplicate charges because of a UI retry.

19. Payment Pending

Trigger: Gateway response is asynchronous/uncertain.

UX: Pending badge → transaction reference → 'We're verifying your payment' → background status refresh.

• Do not ask the customer to pay again immediately.

• Allow leaving the screen while preserving the pending state.

• Notify when the authoritative result is available.

20. Payment Succeeds but Webhook Delayed

Trigger: Gateway has succeeded but RoadResQ webhook/reconciliation has not arrived.

UX: Show payment verification/pending state until server reconciliation confirms success.

• Backend reconciliation is authoritative.

• Do not mark the invoice paid based only on client redirect.

• Once confirmed, update invoice/payment state across all clients.

21. Session Expired

Trigger: Access token/session expires during a flow.

UX: Non-destructive re-authentication → return user to the interrupted screen when safe.

• Do not discard unsaved form data unnecessarily.

• Do not expose protected booking information while logged out.

• After authentication, re-fetch authoritative state.

22. Unauthorized Resource Access

Trigger: User attempts to access a booking/vehicle/provider resource they are not authorized to view.

UX: Generic authorization message → safe navigation back.

• Do not reveal whether another user's resource exists beyond policy.

• Server-side object-level authorization is authoritative.

• Log suspicious repeated attempts for security monitoring.

23. Service Temporarily Unavailable

Trigger: API/dependency outage prevents a feature from operating.

UX: Clear degraded banner/state → explain affected feature → provide alternative if available.

• Core assistance should degrade gracefully where possible.

• Do not present unavailable actions as clickable if they cannot work.

• Provide support/escalation for active jobs when necessary.

24. Maps Service Unavailable

Trigger: Map/routing provider fails or quota is exhausted.

UX: Location/address data remains visible → map shows unavailable state → textual distance/address fallback where possible.

• Do not lose the booking because the map is unavailable.

• Allow manual location display.

• Provider/customer contact remains available if policy permits.

25. Notification Delivery Failure

Trigger: Push/SMS/email notification fails.

UX: Continue based on authoritative booking state; retry notification in background.

• Do not duplicate critical notifications indefinitely.

• Use alternate channel where approved and appropriate.

• Show the information inside the app whenever possible.

26. Realtime WebSocket Disconnect

Trigger: WebSocket connection drops during active booking.

UX: Small connection state indicator → automatic reconnect → authoritative state reconciliation.

• Do not show stale status as current.

• Do not create duplicate events after reconnect.

• Use sequence/event IDs where needed to reconcile missed events.

27. Provider Arrives but OTP Fails

Trigger: Arrival verification code does not match.

UX: Retry with safe limit → alternative verification/support path if policy allows.

• Do not expose sensitive OTP information.

• Rate-limit attempts.

• Escalate repeated failures or suspicious patterns.

28. Estimate Changes After Approval

Trigger: Provider identifies additional work after initial estimate approval.

UX: New work shown separately → reason → incremental cost → updated total → explicit approval.

• Never silently change an already approved amount.

• Customer can approve/decline according to business rules.

• Record approval event and timestamp.

29. Provider Goes Offline During Active Job

Trigger: Provider device loses connectivity after accepting/starting a job.

UX: Customer sees last known status + freshness; provider app queues safe updates for retry where appropriate.

• Booking remains server-authoritative.

• Do not auto-cancel solely because a device is offline.

• Support escalation if the offline period exceeds operational threshold.

30. Booking State Conflict

Trigger: Two clients/actions attempt incompatible booking transitions.

UX: Refresh authoritative state → explain that booking changed → update available actions.

• Server validates allowed transitions.

• Use optimistic UI only when it can safely reconcile.

• Do not show both conflicting states simultaneously.

31. Stale Screen / Browser Back

Trigger: User returns to a screen after booking/payment state changed elsewhere.

UX: Re-fetch authoritative state → show current status → discard stale actions.

• Do not allow an old approval/cancel/pay button to execute against invalid state.

• Show a small 'Updated' message where helpful.

32. App Refresh / Crash During Active Booking

Trigger: App/browser reload or crash while assistance is active.

UX: On launch, detect active booking → restore active booking card → show current authoritative status.

• Do not require the user to recreate the request.

• Restore map/tracking if available.

• Preserve pending support/payment context.

33. No Vehicle Selected

Trigger: User starts assistance without a vehicle.

UX: Explain why vehicle information helps → [Add Vehicle] → optionally allow supported fallback if business rules permit.

Do not silently use the wrong vehicle as the default.

34. Invalid / Unsupported Vehicle

Trigger: Vehicle information cannot be validated or provider does not support the vehicle type.

UX: Explain limitation → edit vehicle → choose another provider/service.

35. Service Outside Provider Radius

Trigger: Selected provider/garage cannot service the location.

UX: Clearly show unavailable status → suggest nearby eligible providers or expand search.

36. Provider Becomes Unavailable During Matching

Trigger: Provider goes offline after appearing eligible.

UX: Remove/expire provider offer → continue matching → avoid showing stale availability.

37. Cancellation During Matching

Trigger: Customer cancels while providers are being contacted.

UX: Confirm → stop dispatch → show cancellation confirmation.

Backend requirement: Cancel must be idempotent and prevent late provider acceptance from creating an active booking.

38. Cancellation Race Condition

Scenario: Customer cancels at the same time provider accepts.

UX: Server resolves authoritative outcome → client receives final state.

• If cancellation wins, provider sees cancelled.

• If acceptance wins and cancellation is no longer permitted, customer sees policy-based outcome.

• Audit both attempts.

39. Double Booking / Multiple Active Requests

Trigger: User attempts multiple emergency requests for the same vehicle/location within a short period.

UX: Detect active assistance → show existing request → [View Active Request] rather than creating another.

Business rules may permit multiple independent scheduled bookings, but emergency duplicate prevention should be strict.

40. Slow Network

Trigger: High latency without complete disconnection.

UX: Preserve UI responsiveness → show contextual progress → prevent duplicate submissions.

• Avoid arbitrary short timeouts that cause false failures.

• Use request timeout messaging only when the client can no longer safely wait.

• After timeout, query server state before retrying a state-changing operation.

41. Offline Mode

What can continue: viewing cached non-sensitive information, composing safe draft data where appropriate.

What must verify online: emergency request creation, provider assignment, booking transitions, estimate approval, payment, cancellation when state-dependent.

Offline UI must clearly distinguish local draft state from server-confirmed state.

42. Invalid Media Upload

Trigger: Unsupported file type, oversized file or failed upload.

UX: Explain accepted format/size → allow replace/retry → preserve other form data.

• Do not expose raw storage errors.

• Scan/validate uploads server-side.

• Show upload progress for larger files.

43. Duplicate Media Upload

Trigger: User taps upload repeatedly or network retry occurs.

UX: Show one upload item with progress/status; deduplicate safely where supported.

44. Invalid Address / Landmark

Trigger: Address search returns ambiguous or no result.

UX: Show suggestions → allow manual pin → request landmark/description if necessary.

Never pretend a low-confidence geocoded address is exact.

45. Customer Changes Location During Matching

Trigger: GPS changes significantly or user moves pin.

UX: Detect change → ask for confirmation → rematch if needed.

• Do not silently dispatch to an outdated location.

• Show updated location before confirmation.

• If provider is already en route, apply business rules before changing destination.

46. Provider Cannot Reach Customer

Trigger: Provider reports access/route problem.

UX: Customer receives clear action request → update landmark/location/contact details → support escalation if unresolved.

47. Customer Does Not Respond

Trigger: Provider arrives or attempts contact without response.

UX: Provider sees contact/escalation procedure; customer receives missed-contact notification.

Any cancellation/fee outcome must follow explicit policy and audit rules.

48. Safety / Emergency Disclaimer State

RoadResQ should clearly state that it is a roadside assistance/service platform and not a replacement for police, ambulance or fire emergency services. When a user reports a potentially life-threatening emergency, the UX should direct them toward appropriate public emergency services while preserving the ability to access roadside assistance when safe.

49. Admin Recovery State

Trigger: Operational exception requires authorized intervention.

UX: Admin sees affected booking → evidence/timeline → allowed recovery actions → confirmation + mandatory reason → audit result.

• Never provide unrestricted arbitrary status editing.

• High-risk actions require elevated permissions.

• Show before/after state clearly.

• Record actor, timestamp, reason and request/correlation ID.

50. Edge-Case Decision Matrix

Scenario	Primary UX	Safe next action
No provider	Explain + alternatives	Retry/expand/garage
Provider cancels	Automatic re-match where possible	Choose replacement/support
GPS unavailable	Manual location	Search/pin
Network lost on submit	Preserve state	Check server then retry
Duplicate request	Show existing	Open active request
Payment fails	No false success	Retry/other method
Payment pending	Verification state	Wait/reconcile
WebSocket lost	Last known + freshness	Reconnect/reconcile
Map unavailable	Textual fallback	Use address/contact
Estimate changes	Incremental approval	Approve/decline
Session expires	Safe re-auth	Return to flow

Scenario	Primary UX	Safe next action
State conflict	Refresh authoritative state	Follow current state
App crash	Restore active booking	Continue
Provider offline	Last known status	Support/escalate

51. UX State Priority Rules

• Safety state overrides convenience.

• Financial integrity overrides speed.

• Server-confirmed state overrides optimistic/local state.

• User-entered data should be preserved whenever safe.

• Active booking continuity overrides generic navigation.

• Security/privacy restrictions override diagnostic convenience.

• A clear degraded experience is better than a misleading normal-looking screen.

52. Edge-Case Testing Requirements

Category	Minimum tests
Matching	No providers, provider rejects, timeout, provider cancels, provider goes offline
Location	GPS denied, inaccurate GPS, manual pin, location changes
Network	Offline before submit, during submit, after submit, slow network, reconnect
Booking	Duplicate create, state conflict, stale screen, cancellation race
Payment	Failure, pending, webhook delay, duplicate attempt, unknown result
Realtime	Disconnect/reconnect, missed events, stale location
Auth	Expired session, unauthorized object, re-auth recovery
Media	Invalid type, oversized, upload failure, duplicate upload
Recovery	App crash/refresh, server restart, dependency outage

53. Analytics & Observability Events

• edge_no_provider

• edge_provider_cancelled

• edge_gps_unavailable

• edge_network_lost

• edge_duplicate_request_detected

• edge_payment_failed

• edge_payment_pending

• edge_realtime_disconnected

• edge_map_unavailable

• edge_state_conflict

• edge_session_expired

Telemetry must avoid sensitive free-text, credentials and unnecessary precise location data. Edge-case events should support diagnosis without exposing private information.

54. Support Escalation Rules

Escalate when	Reason
Active customer safety concern	Immediate operational attention
Payment status cannot be reconciled	Financial integrity
Booking state is contradictory	Data integrity
Provider/customer identity mismatch	Trust/safety
Repeated location/tracking failure	Operational continuity
Suspected fraud/security issue	Security investigation
No provider during urgent need	Customer impact
System-wide dependency outage	Platform incident

55. Definition of Done — Edge Cases

• Critical customer, provider and admin flows have defined failure states.

• No-provider and provider-cancellation flows have recovery paths.

• GPS and network failure do not unnecessarily destroy active requests.

• Duplicate booking/payment actions are protected by idempotency.

• Payment states distinguish failed, pending, success and unknown outcomes.

• Realtime loss shows freshness/degraded state and reconciles on reconnect.

• Booking state conflicts resolve to authoritative server state.

• Emergency/safety limitations are clearly communicated.

• Every critical edge case has QA coverage and an operational response.

56. Final UX Resilience Standard

RoadResQ must be designed for the real world, not only the ideal path. Roads have poor connectivity, GPS can fail, providers can cancel, payments can become uncertain and users can tap twice. A production-quality RoadResQ experience protects the user's intent, preserves authoritative state, prevents duplicate financial or booking effects, clearly communicates uncertainty and always provides the safest available next step.